Prompt:

- think and devide the problem into simple tasks then search the internet and then thing again give me the steps to build this project i know the openai sdk and crewai framework and all you know about me
- File AI_readiness_ass.pdf

Output:

Here is a clean way to break the assessment into simple build tasks, then turn it into a solid project plan.

The assessment is a month-end reconciliation problem: you have platform transactions and bank settlements, and your job is to find why they do not match. The brief also says you must generate your own test data, deliberately plant four gap types, state your assumptions, and submit the full brainstorming/thread/output trail.

1) Split the problem into small tasks

First, make the problem mechanical instead of “AI-ish”:

Task A — define the data model
Create one schema for platform transactions and one for bank settlements.
Minimum fields: `transaction_id`, `order_id`, `amount`, `currency`, `timestamp`, `status`, `customer_id`, `settlement_date`, `source_system`.

Task B — generate synthetic data
Build a synthetic dataset that looks realistic and includes the four required anomalies from the brief:

1. a transaction settled in the following month
2. a rounding difference visible only in aggregation
3. a duplicate entry in one dataset
4. a refund without a matching original transaction

Task C — normalize and clean
Standardize timestamps, currency precision, ID formats, and status labels before matching.

Task D — reconcile records
Match platform rows to bank rows using deterministic rules first:

- exact ID match if available
- otherwise fuzzy match on amount + date window + reference fields
- allow settlement lag of 1–2 days, and month crossover cases

Task E — detect gaps
Write rule-based detectors for:

- unmatched platform transactions
- unmatched bank settlements
- duplicates
- refunds without originals
- rounding-only mismatches
- late settlements crossing month boundary

Task F — explain each gap
For every anomaly, generate a short explanation:

- what happened
- why it happened
- which rows prove it
- whether it is a true exception or an expected timing issue

Task G — produce output
Generate:

- a reconciliation summary
- anomaly table
- month-end gap report
- assumptions list
- test cases
- final demo output

2) Best project structure

Build it as a pipeline, not as one giant prompt.

Backend flow

1. **Input generator**
   Produces synthetic platform and bank datasets.
2. **Preprocessor**
   Cleans and normalizes records.
3. **Matcher**
   Creates candidate pairs between platform and bank rows.
4. **Exception engine**
   Flags mismatches and classifies them.
5. **LLM explanation layer**
   Turns detected anomalies into a readable business report.
6. **Report exporter**
   Saves CSV/JSON/HTML/PDF outputs.

3) How to use OpenAI SDK and CrewAI in this project

CrewAI is a good fit because it supports **agents**, **tasks**, **structured outputs**, **flows**, and even **human input** when a step is ambiguous. (CrewAI Documentation)

OpenAI's current docs also emphasize **structured outputs** and **function calling**, and recommend the **Responses API** for new projects. They also note that using **pinned model versions** plus **evals** is the best way to keep behavior stable over time. (OpenAI Platform)

So the cleanest design is:

Agent 1: Data Assumption Agent

Role: decide reconciliation assumptions.
Output: JSON assumptions like:

- settlement window = 2 days
- month-end cutoff = last day 23:59:59
- rounding tolerance = 0.01
- duplicate definition = same order_id + same amount + same timestamp window

Agent 2: Reconciliation Analyst

Role: match records and detect gaps.
Output: structured anomaly list.

Agent 3: Business Explainer

Role: convert technical anomalies into plain-English findings.
Output: final report.

Optional Agent 4: QA / Test Agent

Role: verify that all four planted anomaly types are actually detected.

4) Recommended build order

Step 1 — lock the schema

Define two Pydantic models:

- `PlatformTransaction`
- `BankSettlement`

Then define:

- `ReconciliationResult`
- `AnomalyRecord`
- `AssumptionSet`

This matters because structured outputs are much easier to validate than free-form text. (CrewAI Documentation)

Step 2 — write synthetic data generation

Generate about 100–500 rows first.

Include:

- normal matched payments
- late settlements
- duplicate bank rows
- rounding edge cases
- refund-only rows

Keep a hidden “ground truth” file so you know which anomalies were planted.

Step 3 — build deterministic reconciliation logic

Do not start with the LLM.

Use code to:

- match exact IDs
- match by order_id and amount
- allow settlement delay
- group by month
- compare sums
- detect duplicates
- detect orphan refunds

This gives you a reliable core and makes the LLM layer much safer.

Step 4 — add anomaly classification

Classify each mismatch into one of these buckets:

- timing difference
- duplicate

- rounding variance
- missing original
- missing settlement
- unknown exception

Step 5 — add LLM explanation

Use OpenAI only for:

- summarizing anomalies
- generating business-friendly reasoning
- drafting the final report
- producing the brainstorming and distilled prompt artifacts

Keep the actual matching logic in code.

Step 6 — wrap it in CrewAI

Use CrewAI to orchestrate the workflow:

- agent 1 sets assumptions
- agent 2 runs reconciliation reasoning
- agent 3 writes the report
- agent 4 checks completeness

Step 7 — create tests

Write test cases for each required anomaly:

- settlement next month
- rounding sum mismatch
- duplicate row
- refund without original
- perfect-match control case
- all-anomalies-together case

Step 8 — generate submission artifacts

The PDF says the final submission should include:

- brainstorming thread
- distilled prompt
- full execution thread
- test cases
- working output with code and demo, plus three production failure notes

So make your system save:

- prompt logs
- agent traces
- anomaly JSON
- final markdown report

5) A simple architecture that will work well

Recommended stack

- Python
- pandas
- Pydantic
- OpenAI SDK
- CrewAI
- FastAPI or Streamlit for demo
- pytest for tests
- SQLite or CSV for persistence

Pipeline

1. generate synthetic input
2. run rules engine
3. send unresolved cases to LLM
4. produce final report
5. export evidence

6) What your final output should look like

Your final report should have sections like:

- Summary of month-end mismatch
- Total platform amount
- Total bank amount
- Net difference
- Number of matched transactions
- Number of unmatched transactions
- Detected duplicates
- Late settlements crossing month boundary
- Refunds without originals
- Rounding differences
- Assumptions

7) What to avoid

Do not let the LLM do raw record matching by itself.
Do not skip assumptions.
Do not hide the synthetic-data generation.
Do not treat all mismatches as errors; some are timing effects.
Do not produce only a chatbot answer; the assessment wants a working analytical system plus artifacts.

8) Best implementation strategy for you

Since you already know OpenAI SDK and CrewAI, the fastest strong version is:

Phase 1: pure Python reconciliation engine
Phase 2: CrewAI orchestration around the engine
Phase 3: OpenAI-generated narrative and audit trail
Phase 4: polished demo UI

That gives you something robust, explainable, and easy to defend in a review.

If you want, I can turn this into a full folder structure with file names and a step-by-step implementation checklist.

Prompt to Claude ai:

- think how to impliment this project then search the internet and then think again i want different file of python that handel different parts of the project, like a package i do in different projects also this.these are my current works you can use the information from here and then think search multiple times to build this project test code also by renning use latest stable python packages only

Instrction Prompt :

- Build a complete Python solution for a month-end reconciliation problem where transactions and bank settlements do not match. No input data is provided, so generate realistic synthetic datasets.
-
- ## Core Tasks
-
- * Generate transactions & settlements
- * Reconcile both datasets
- * Detect anomalies

- * Handle month-end timing gaps
- * Explain each anomaly clearly
- * Output clean Markdown + structured JSON
- * Run everything from a single entry file
-
- ## Mandatory Anomalies
-
- * Next-month settlement
- * Rounding difference (aggregate level)
- * Duplicate entry
- * Refund without original transaction
-
- ## Outputs
-
- * Reconciliation summary
- * Anomaly table
- * Month-end gap report
- * Assumptions
- * Test cases
- * Final demo output
-
- ## Testing (Strict)
-
- * Write tests for matching, anomalies, and edge cases
- * Run tests before final output
- * Fix failures → re-test → then generate results
- * Do not produce outputs if tests fail
-
- ## LLM Support (OpenAI / Ollama)
-
- * Support both OpenAI API and Ollama (http://localhost:11434)
- * Configurable model (llama3, mistral, etc.)
- * Wrapper for LLM calls
- * Try Ollama first → fallback to OpenAI
- * Output must include:
-
- * What happened
- * Why it happened
- * Evidence
- * Expected vs real issue
- * Use structured prompts, low temperature
-
- ## Requirements

-
- * Modular `.py` files (no fixed structure)
- * Clean, readable, production-quality code
- * Define assumptions (timing, rounding, etc.)
- * Save:
-
-    * anomalies.json
-    * reconciliation_summary.json
-    * gap_report.json
-    * final_output.json
-
- ## Goal
-
- One-command execution that:
-
- * Passes all tests
- * Generates accurate reconciliation insights
- * Produces clean Markdown + JSON
- * Clearly explains all anomalies
-
- Now generate the full implementation.
-

Output :

Main
PY
Download

Config
PY
Download

Data gen
PY
Download

Cleaner
PY
Download

Reconciler
PY
Download

Gap detector
PY
Download

Explainer
PY
Download

Reporter
PY
Download

init
PY
Download

Test data gen
PY
Download

Test reconciler
PY
Download

Test gap detector
PY
Download

Test reporter
PY
Download

Reconciliation report
Document · MD
Download

Sonnet 4.6 Extended

Claude is AI and can make mistakes. Please double-check responses.

```python
—
    log.info("── [7/7] Saving outputs to %s/ ──", args.output_dir)
    llm_demo = _build_demo_narrative(summary, anomaly_df, gap_rpt_df, cfg.llm)

    paths = save_outputs(
        txn_df, sett_df, recon, gaps,
        gap_explanations=gap_explanations,
        llm_demo_text=llm_demo,
        cfg=cfg.output,
    )

    # ── Console summary ──────────────────────────────────────────────────────
    _print_summary(summary, anomaly_df, gap_rpt_df)

    print("── Output Files ──")
    for name, path in paths.items():
        print(f"  {name:<28}  {path}")

    if llm_demo:
        print("\n── Executive Summary (LLM) ──")
        print(textwrap.fill(llm_demo, width=70, initial_indent="  ", subsequent_indent="  "))

    print("\n✅  Pipeline complete.\n")


# ---------------------------------------------------------------------------
# Entry point
# ---------------------------------------------------------------------------

def main() -> None:
    args = parse_args_or_defaults()

    if not args.skip_tests:
        ok = run_tests()
        if not ok:
            sys.exit(1)

    run_pipeline(args)


def parse_args_or_defaults() -> argparse.Namespace:
    """Wrapper so main() can be called without CLI args in tests."""
    try:
        return _parse_args()
    except SystemExit:
        # Called programmatically — use defaults
        return argparse.Namespace(
            no_llm=False, skip_tests=False,
            n=5_000, seed=42, output_dir="outputs"
        )


if __name__ == "__main__":
    main()
```